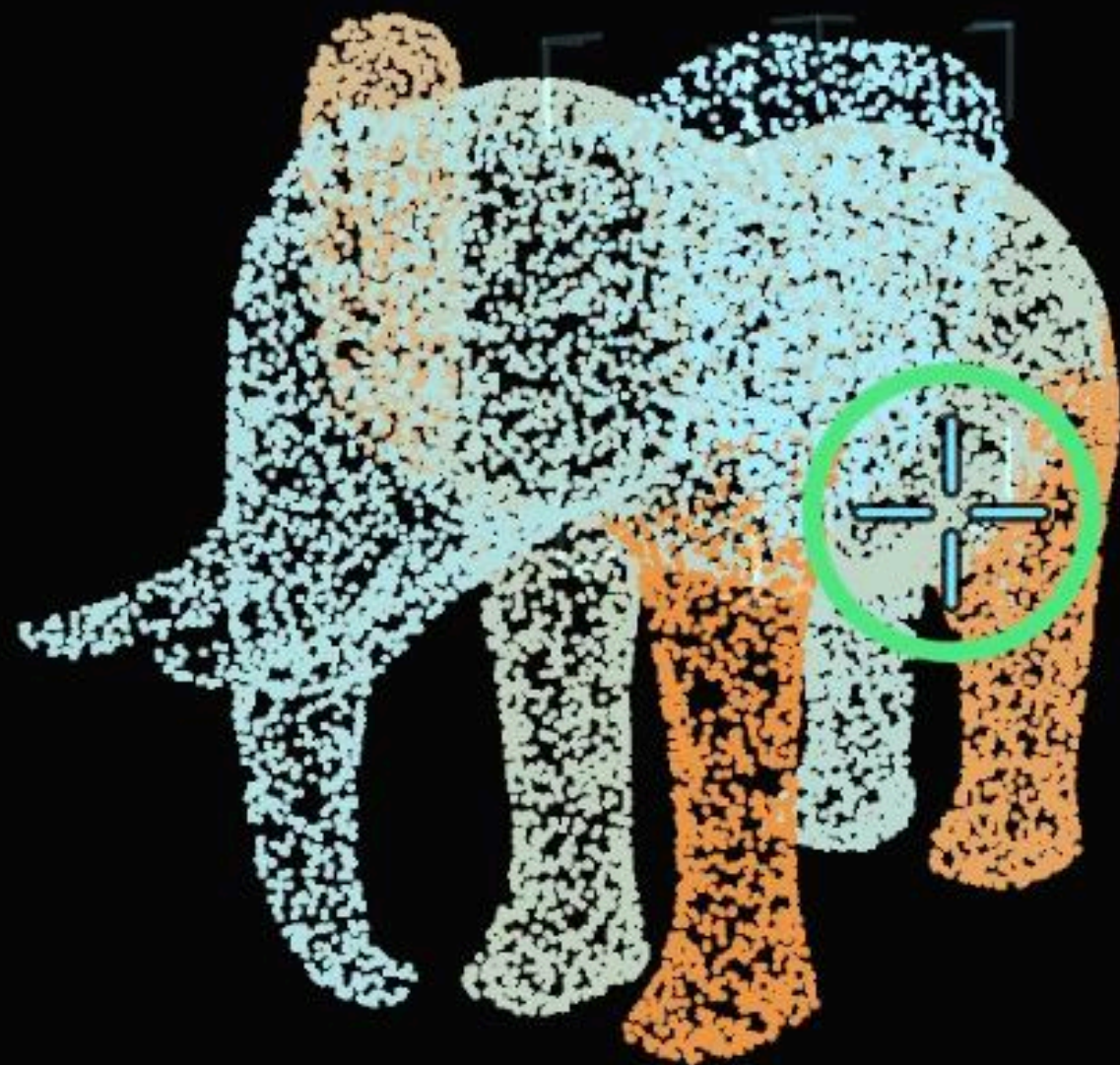
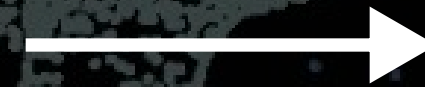


Blind Men & the Elephant

Computer Graphics 26-1 Project 9
AxT 20241113 Giwan Yoon



Concept



p5.js

Ray Casting
Morphing

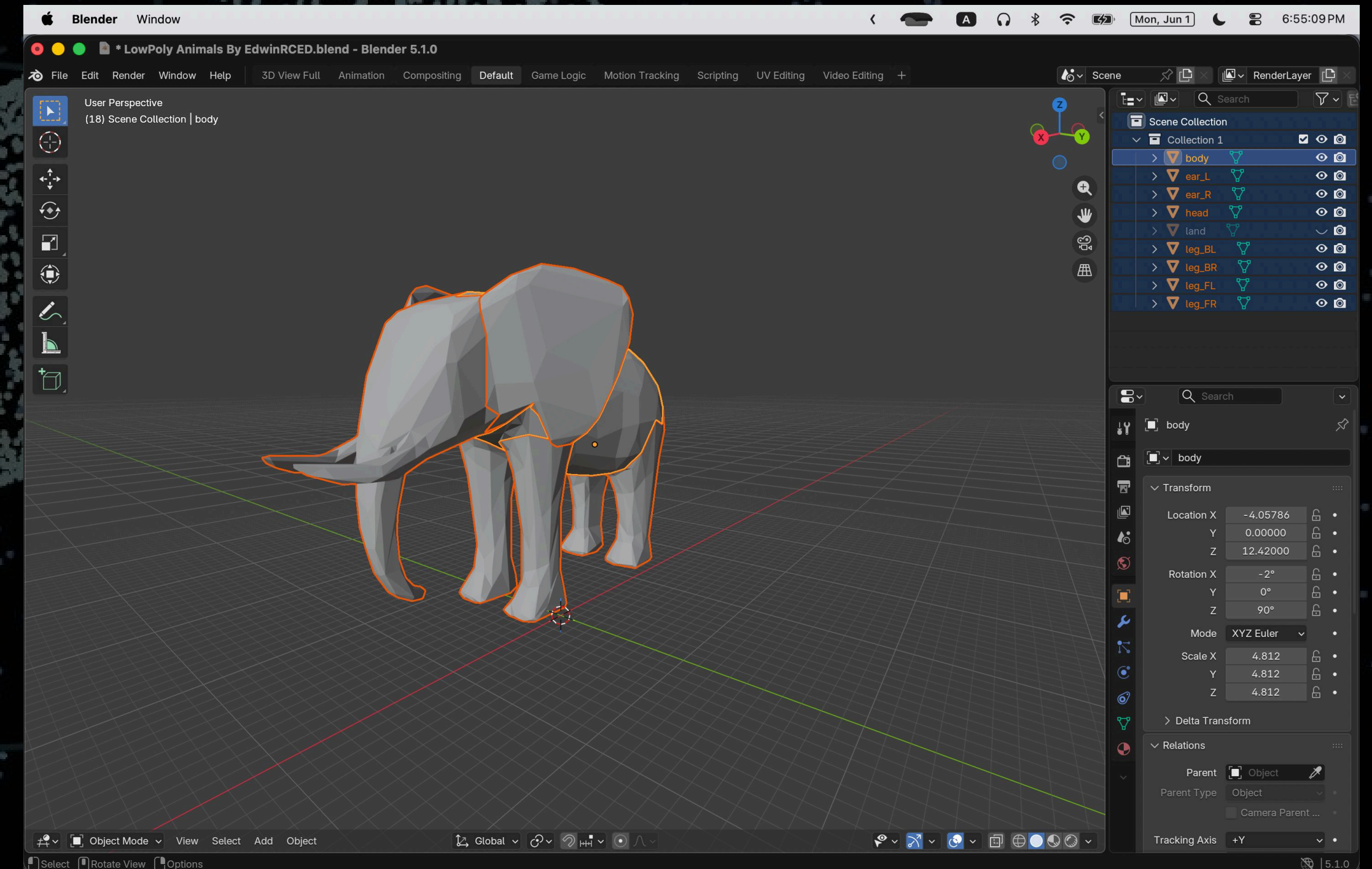
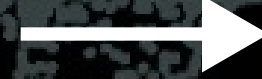
... a story of a group of blind men who have never come across an elephant before and who learn and imagine what the elephant is like by touching it.

from wikipedia

Process



3D model from <https://skfb.ly/6D7xl>



Divide part in Blender



OBJ file for each part

Process

부위별 OBJ 로딩

코끼리를 8개 부위로 분리해 각각 로드한다. `loadModel` 은 단일 파일의 그룹을 하나로 합쳐버려 부위 식별이 불가능하기 때문. 동일한 transform 아래에서 함께 처리하면 한 마리로 정렬된다.

```
const PARTS = ['head', 'body', 'ear_L', 'ear_R', 'leg_FL', 'leg_FR', 'leg_BL', 'leg_BR'];
function preload() {
  for (const name of PARTS) parts[name] = loadModel(`src/elpt/${name}.obj`); // normalize 끄고 좌표 정렬 유지
}
// 로컬 → 월드 (draw의 transform 순서와 반드시 동일)
function modelToWorld(v) {
  let p = v.copy().mult(15); // scale(15)
  p = createVector(p.z, p.y, -p.x); // rotateY(HALF_PI)
  p = createVector(p.x, -p.y, -p.z); // rotateX(PI)
  return p.add(20, 40, -20); // translate
}
```

Process

부위별 충돌 검사 (Ray-AABB, slab)

각 부위의 월드 AABB를 미리 계산하고, 카메라에서 시선 방향으로 쏜 ray와 **슬랩** 알고리즘으로 교차한다. 가장 가까운(t 최소) 부위가 조준 대상.

```
function rayAABB(ro, rd, bmin, bmax) {  
  const t1 = (bmin.x - ro.x) / rd.x, t2 = (bmax.x - ro.x) / rd.x; // y, z축도 동일  
  const tmin = max(max(min(t1, t2), min(t3, t4)), min(t5, t6));  
  const tmax = min(min(max(t1, t2), max(t3, t4)), max(t5, t6));  
  if (tmax < 0 || tmin > tmax) return Infinity; // 미충돌  
  return tmin >= 0 ? tmin : 0; // 박스 안이면 0  
}  
// pickPart: 모든 부위에 대해 rayAABB의 t를 구해 가장 작은 부위를 반환
```

Process

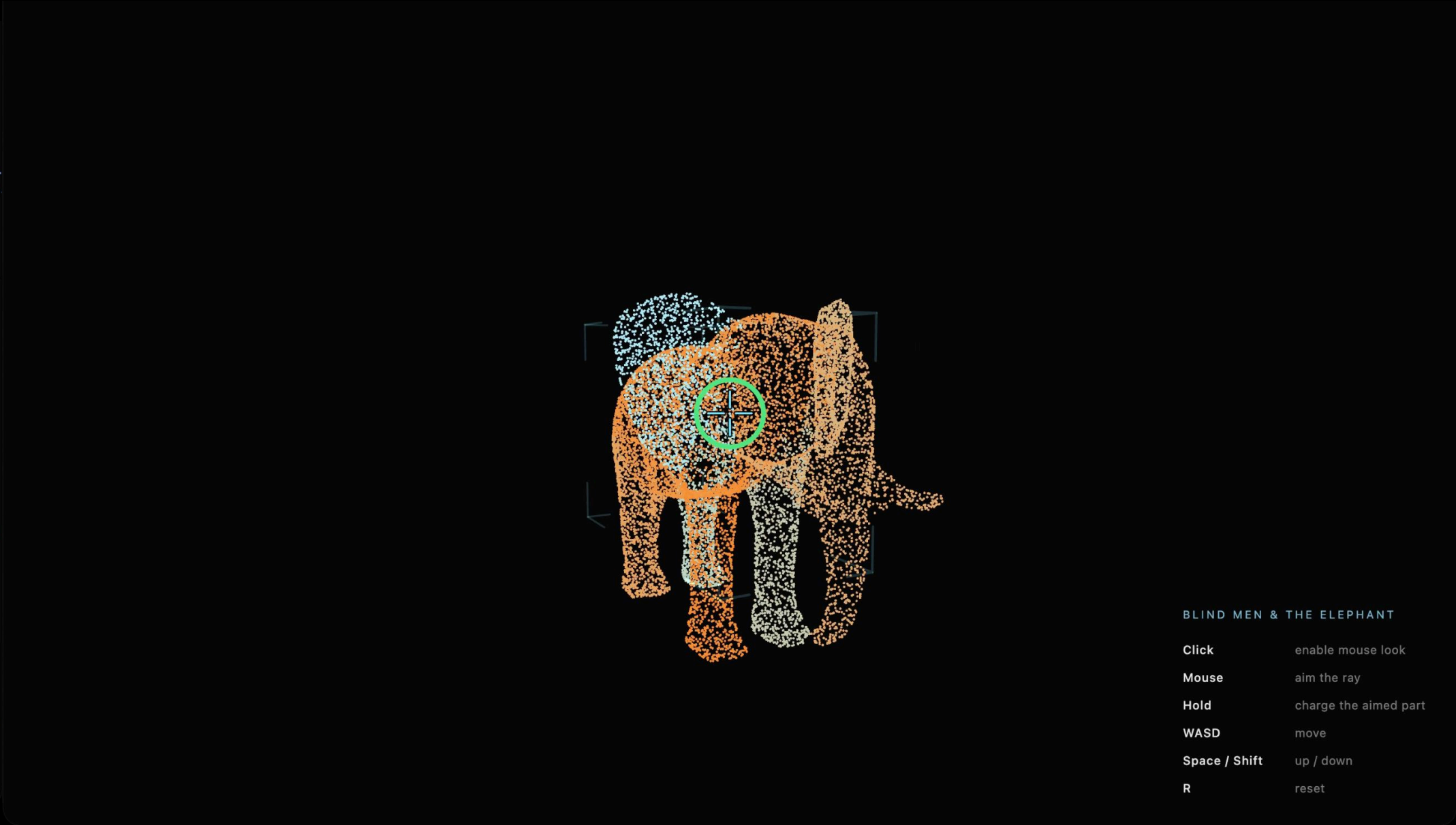
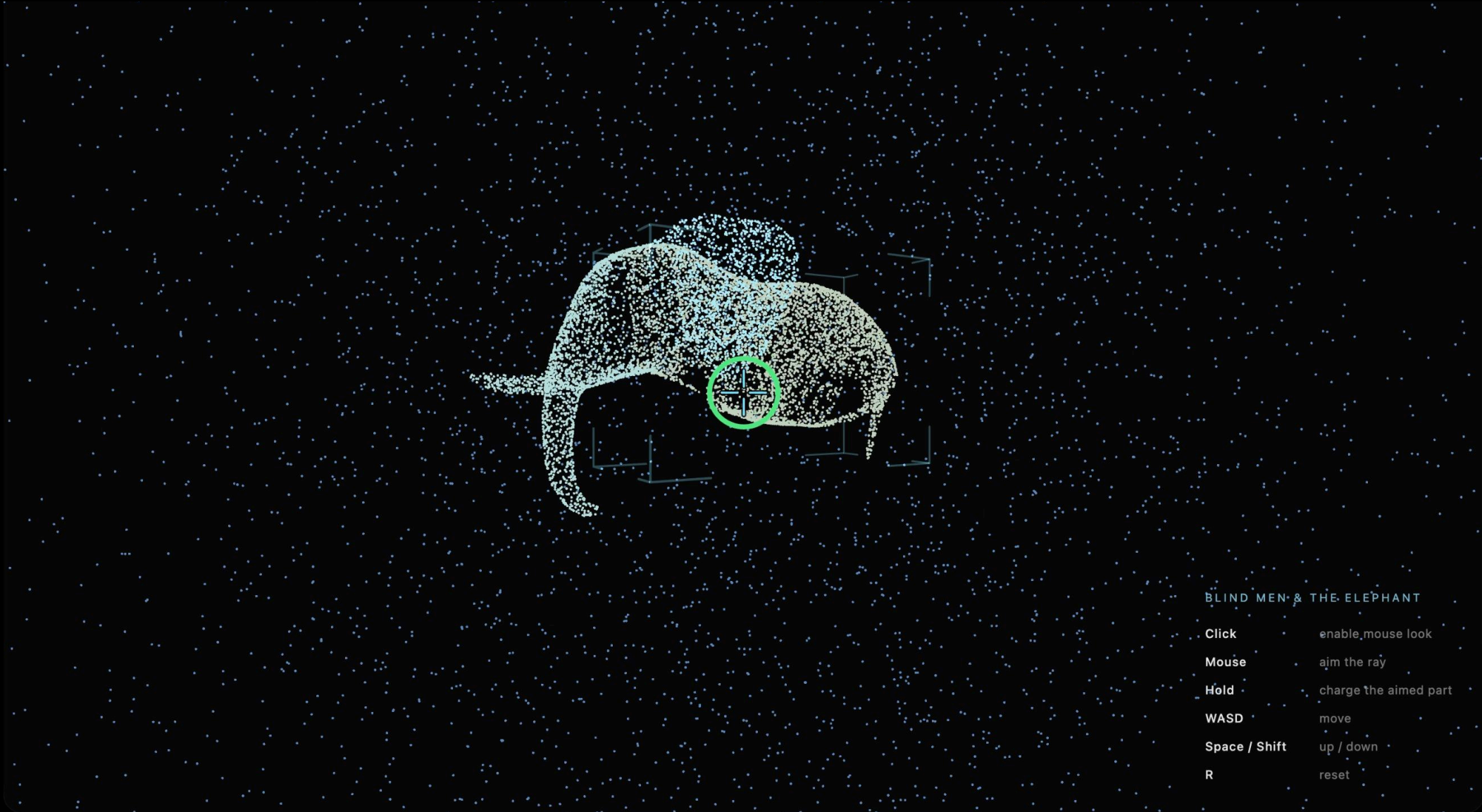
차징 기반 모핑

조준한 부위만 `charge` 가 0→1로 차오르고, 가득 차면 `HOLD_TIME` 동안 고정,
아니면 방전된다. 점 위치는 `charge` 로 보간한다.

```
// 상태 갱신
if (hold[name] > 0)          { hold[name] -= dt; charge[name] = 1; } // 완성 후 고정
else if (locked && mouseIsPressed && name === hit) {                // 조준 + 홀드 → 충전
    charge[name] = min(1, charge[name] + dt / CHARGE_TIME);
    if (charge[name] >= 1) hold[name] = HOLD_TIME;
} else charge[name] = max(0, charge[name] - dt / DECHARGE_TIME); // 방전(분산)

// 렌더: 흩어진 위치(base + 드리프트) → home(표면) 으로 charge만큼 보간
vertex(lerp(driftX, pt.home.x, charge[name]) /* y, z 동일 */);
```

Result



[link](#)
[video](#)

